

model/cread.py (Q&A)

Q: What is the purpose of model/cread.py? **A:** It implements the CREAD baseline model used in this repo. The model maps feature dictionaries to a vector of per-bucket probabilities, intended to work with the discretization/reconstruction strategy implemented in `run_cread.py`.

Q: What does the model output and what does it represent? **A:** `forward(x_dict)` outputs a tensor of shape `[batch, head_num]` after a sigmoid, where each head corresponds to a discretized threshold/bucket and is trained with a binary cross-entropy objective in the run script.

Q: How are features encoded? **A:** Similar to other models, it uses embeddings for sparse and sequence features (sequence pooled with a mask) and linear layers for continuous features; the pooled embeddings are fed into a shared MLP to produce a shared representation.

Q: How does the “multi-head” output work? **A:** After computing the shared representation, CREAD applies a separate small MLP per head (stored in `output_mlps`), concatenates their outputs, and applies sigmoid to obtain per-head probabilities.

Q: Code example: multi-head construction pattern? **A:**

```
self.output_mlps = torch.nn.ModuleList()
for _ in range(head_num):
    self.output_mlps.append(MultiLayerPerceptron(..., output_layer=True))
```

Q: Full source code? **A:**

```
import torch

from model.layers import FactorizationMachine, MultiLayerPerceptron

class Cread(torch.nn.Module):

    def __init__(self, description, embed_dim, share_mlp_dims, output_mlp_dims, head_num,
                 dropout):
        super().__init__()
        self.features = {name: (size, type) for name, size, type in description if (type
                                          in ["ctn", 'seq', 'spr'])}
        self.build(embed_dim, share_mlp_dims, output_mlp_dims, head_num, dropout)

    def build(self, embed_dim, share_mlp_dims, output_mlp_dims, head_num, dropout):
        self.emb_layer = torch.nn.ModuleDict()
        self.ctn_emb_layer = torch.nn.ParameterDict()
        self.ctn_linear_layer = torch.nn.ModuleDict()
```

```

embed_output_dim = 0
for name, (size, type) in self.features.items():
    if type == 'spr':
        self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
        embed_output_dim += embed_dim
    elif type == 'ctn':
        self.ctn_linear_layer[name] = torch.nn.Linear(1, 1, bias=False)
    elif type == 'seq':
        self.emb_layer[name] = torch.nn.Embedding(size, embed_dim)
        embed_output_dim += embed_dim
    else:
        raise ValueError('unkown_ feature_ type: {}'.format(type))
self.share_mlp = MultiLayerPerceptron(embed_output_dim, share_mlp_dims, dropout,
    output_layer=False)
self.output_mlps = torch.nn.ModuleList()
for idx_head in range(head_num):
    self.output_mlps.append(MultiLayerPerceptron(share_mlp_dims[-1],
        output_mlp_dims, dropout, output_layer=True))
return

def init(self):
    for param in self.parameters():
        torch.nn.init.uniform_(param, -0.01, 0.01)

def forward(self, x_dict):
    linears = []
    embs = []
    for name, (_, type) in self.features.items():
        x = x_dict[name]
        if type == 'spr':
            embs.append(self.emb_layer[name](x).squeeze(1))
        elif type == 'ctn':
            linears.append(self.ctn_linear_layer[name](x))
        elif type == 'seq':
            seq_emb = self.emb_layer[name](x)
            seq_mask = torch.unsqueeze(x_dict["{}_mask".format(name)], dim=2)
            embs.append(torch.sum(seq_emb * seq_mask, dim=1) / torch.sum(seq_mask, dim
                =1))
        else:
            raise ValueError('unkwon_ feature: {}'.format(name))
    emb = torch.concat(embs, dim=1)
    share_vec = self.share_mlp(emb)
    output_list = []
    for output_mlp in self.output_mlps:
        output_list.append(output_mlp(share_vec))
    return torch.sigmoid(torch.concat(output_list, dim=1))

```